

Code-Layout, Readability and Reusability

Date	15 July 2026
Team / Submission ID	
Project Name	Emotion Detection & Learning Support Engine
Submitted By	Venu Gopal (Sole Team Member / Team Lead)
Maximum Marks	5 Marks

Code Layout Checklist

This checklist reflects an honest review of the actual repository (app.py + src/ modules).

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Partial — logic is correctly indented, but app.py uses unusually heavy blank-line spacing between statements which hurts readability.
2	Proper File Structure	Files and folders are logically organized	Yes — src/ separates preprocessing, prediction (bilstm/bert), logging, Gemini helper and utils from the app.py UI layer.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes — names like emotion, confidence, probabilities, bilstm_result are clear and self-descriptive.
4	Function / Method Names	Functions are descriptively named	Yes — predict_bert, predict_bilstm, get_gemini_response, save_to_csv, preprocess_text are all descriptive.
5	Code Comments	Inline and block comments explain logic	Partial — section banners (e.g. "# BiLSTM Prediction") exist, but individual functions lack docstrings/inline comments in app.py.
6	Modular Design	Code is split into reusable functions/modules	Yes — 8 focused modules under src/, each with a single responsibility.
7	No Redundant Code	Duplicate or unused code is removed	Partial — app.py imports predict_bilstm and EMOTION_RESPONSES twice (lines 3-7), and the sidebar CSV counter reads a different filename than the logger writes to.
8	Error Handling	Exceptions and errors are handled gracefully	Partial — predictions and the Gemini call are wrapped in try/except, but several except blocks are broad (bare Exception) rather than catching specific errors.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	src/preprocessing.py (preprocess_text, EMOTION_RESPONSES)	Python / NLTK	Used by bilstm_predict.py and app.py fallback logic	High

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
2	src/bilstm_predict.py	Python / TensorFlow-Keras	Called directly from app.py for every analysis request	High
3	src/bert_predict.py	Python / PyTorch, Transformers	Called directly from app.py alongside the BiLSTM prediction	High
4	src/gemini_helper.py (build_prompt, get_gemini_response)	Python / google-generativeai	Used for both the initial response and the "Regenerate Response" action	High
5	src/logger.py, src/utlis.py, src/mixed_emotion.py, src/prediction_schema.py	Python / pandas	Logging, mixed-emotion helpers and the unified prediction schema, reusable across any future interface (API/CLI) beyond Streamlit	Medium

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear module separation; formatting spacing in app.py could be tightened.
Readability	3.5	Descriptive names help a lot; more docstrings and inline comments would improve it further.
Reusability	4	Prediction and helper modules are cleanly decoupled from the Streamlit UI and could be reused in a different frontend or API.
Documentation / Comments	2.5	No README in the repository yet; recommend adding one with setup and architecture notes.
Overall Score	3.5 / 5	A solid, modular codebase for a solo project; main gaps are documentation and a couple of small inconsistencies (see Known Issues in Project Executable Files).